COMP 4300
Intro to Game Programming

Lecture 8
Engine Architecture Upgrades
SFML Textures / Sprites
Basic Texture Animation

A2 Engine Architecture

GameEngine
- > Systems
- > EntityManager
  - > Entity
    - > Component

Components

- Pure Data
- Stored in Entity
- Each Component is its own class

CTransform

```
+ pos:      Vec2f
+ velocity: Vec2f
+ scale:    Vec2f
+ angle:    float
```

CCollision

```
+ radius:   float
```

CScore

```
+ score:    int
```

CShape

```
+ circle: sf::Circle
```

CLifespan

```
+ lifespan:  int
+ remaining: int
```

CInput

```
+ up:    int
+ down:  int
+ left:  int
+ right: int
+ shoot: int
```

Entity

- Entity = Game Object
- Stores Component Tuple
- Tag = Entity ‘type’
- Active = Alive or Dead
- ID = Integer identifier

Entity	
- m_components:	CTuple
- m_alive:	bool
- m_id:	int
- m_tag:	string
+ get<T>:	T&
+ add<T>:	void
+ has<T>:	bool
+ id:	int
+ isAlive:	bool
+ tag:	string&
+ destroy:	void

EntityManager

- Entity ‘Factory’ Class

EntityManager	
- m_entities:	EntityVec
- m_entityMap:	EntityMap
- m_toAdd:	EntityVec
- m_totalEntities:	int
- init():	void
+ update():	void
+ addEntity(tag):	sp<Entity>
+ getEntities():	EntityVec&
+ getEntities(tag):	EntityVec&

- Delayed Entity Add()
  - No Iterator Invalidation
- Secondary map from tag->entity
  - Trade storage for convenience / run time
- Can do other book-keeping like mem mgmt

Game

- Top-Level Game Object
- Holds all game data
- All game system functions
- All gameplay code

Game
- m_window: sf::Window - m_entities: EntityManager - m_paused: bool - m_running: bool
- init: void + update: void // systems - sMovement: void - sUserInput: void - sEnemySpawner: void - sCollision: void - sRender: void

Vec2

- 2D Math Structure
  - see previous lecture
- Templated Class
  - T = (x,y) variable type

```
using Vec2f = Vec2<float>;
```

Vec2<T>
+ x: T + y: T
+ operator ==: bool + operator !=: bool + operator +: Vec2 + operator -: Vec2 + operator *: Vec2 + operator /: Vec2 ... + normalize: void + length: float

What can this engine do?

- Create game objects as Entity instances
- Add Component data to Entities
- Implement game play via Systems
  - Also, handle user input
- Pause game play / exit game
- Can init / load configuration from file

What are the limitations?

- Can only display one ‘scene’
- Can NOT load texture / sounds assets
- Can NOT display textured animations
- Does not have any menu / interface

Game Scene

- A game can contain many different ‘scenes’ that have very different logic / controls
- Example: RPG Games
  - Menu / Text / Dialogue Scene
  - World-Map Scene
  - Combat / Battle Scene
- How can our architecture allow for different game states?

Assignment 2: Game Class

- The Game class from Assignment 2 handled all functionality for our game
- Some of this functionality will remain the same over all game scenes
  - Main Loop Structure, Game Window, Config, Assets
- Some of this functionality will be specific to the type of game scene currently shown
  - Input Controls, Rendering, Game Logic, Collisions

A3: GameEngine / Scenes

- Let’s separate the previous Game class into two classes which handle this new functionality
- GameEngine Class
  - Functionality present in all scenes
  - Construction / Management of Scene(s)
  - Running / Quitting Game
- Scene Class
  - Functionality specific to each scene
  - Entities relevant to that scene

GameEngine Class

- Stores top level game data
  - Assets, sf::Window, Scenes
- Performs top level functionality
  - Changing Scenes, Handling Input
- Run() contains game main loop
- GameEngine pointer will be passed to Scenes in constructor
  - Access to: assets, window, scenes

GameEngine
- m_scenes: map<str,Scene> - m_window: sf::Window - m_assets: Assets - m_scene: string - m_running: bool
- init(): void + update(): void + run(): void + update(): void + quit(): void + changeScene<T>(): void + getAssets(): Assets& + window(): sf::Window& + sUserInput(): void

Scene Base Class

- Stores common Scene data
  - Entities, Frame Count, Actions
- Scene-specific functionality is carried out in Derived classes
- Base Scene class is Abstract, cannot be instantiated
- simulate() calls derived scene's update() a given number of times (will be useful later)

Scene (Abstract)
- m_game: GameEngine* - m_entities: EntityMgr - m_frame: int - m_actionMap: map<int,str> - m_paused: bool
+ update(): void=0 + sDoAction(action): void=0 + sRender(): void=0 + simulate(int): void + doAction(action): void + registerAction(a): void

Scene Derived Class

- Stores Scene-specific data
  - Level, Player Pointer, Config
- Scene-specific Systems are defined within the derived class
- Some Scene derived classes may require quite different systems based on functionality
- update(), sRender(), sDoAction() must be implemented for each Scene

Scene_Play : Scene	
- m_level:	string
- m_config:	PlayerConfig
- init():	void
+ update():	void
- sAnimation():	void
- sMovement():	void
- sEnemySpawner():	void
- sCollision():	void
- sRender():	void
- sDoAction(action):	void
- sGUI():	void

Scene Switching

- GameEngine stores a map from strings to `shared_ptr<Scene>`
- Also stores a currentScene string
- `currentScene()` looks up the currently active scene by `map[currentScene]`
- `changeScene(string, scene)` changes the scene to a new, or previously stored Scene
- Mimics a finite state machine for Scene switching

GameEngine
- m_scenes: map<str,Scene> - m_window: sf::Window - m_assets: Assets - m_scene: string - m_running: bool
- init(): void + update(): void + run(): void + update(): void + quit(): void + changeScene<T>(): void + getAssets(): Assets& + window(): sf::Window& + sUserInput(): void

Scene Switching Example

GameEngine
- m_scenes: map<str,Scene> - m_window: sf::Window - m_assets: Assets - m_scene: string - m_running: bool
- init(): void + update(): void + run(): void + update(): void + quit(): void + changeScene<T>(): void + getAssets(): Assets& + window(): sf::Window& + sUserInput(): void

1. Game Engine constructed:
   - changeScene<T>("string", args)
     - currentScene = "menu"
     - scenes["menu"] = sp<Scene_Menu>(args)
2. Player presented with menu scene
3. Player selects a level on the menu
   - LevelPath = currently selected menu item
   - Menu tells game engine to change scenes
   - game->changeScene<Scene_Play>("play", LevelPath)

Asset Management

- Assets are external files that are loaded into memory to be used in the game
- For our games, assets will be:
  - Textures (image files: .png, .jpg)
  - Animations (textures + bookkeeping)
  - Sounds (sound files: .wav, .ogg)
  - Fonts (filetype: .ttf)
- “Load once, use often”

Assets Class

- Want to load assets that are defined in an external configuration file
  - `Texture MegaMan textures/MMS.png`
  - `Sound MegaDeath sounds/death.wav`
- We can then reference that asset via name:
  - `m_assets->getTexture("MegaMan");`
  - `m_assets->getSound("MegaDeath");`
- To implement this, the Asset class will use a `std::map<std::string, AssetType>`

Assets Class

- Lives inside the GameEngine class
- Initialized inside GameEngine init()
- Accessed via Scene's pointer to GameEngine
  - `m_game->getAssets()`

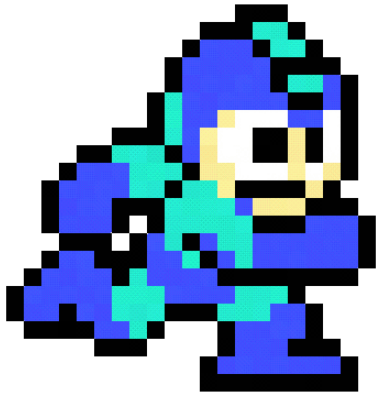
Assets
- m_textures: map<str,sf::Texture> - m_animations: map<str, Animation> - m_sounds: map<str, sf::Sound> - m_fonts: map<str, sf::Font>
+ addTexture(name, path): void + addSound(name, path): void + addFont(name, path): void + addAnim(name, Animation): void + getTexture(name): sf::Texture& + getSound(name): sf::Sound& + getFont(name): sf::Font& + getAnimation(name): Animation&

Assets Class

- Assets loaded once at beginning of program
- Assets can then be used in any scene, but they are only initialized once in memory
- For larger games, you may want to load assets per scene
- For example, loading screens on level change in most games are for Asset management and other data loading purposes

Assets
- m_textures: map<str,sf::Texture> - m_animations: map<str, Animation> - m_sounds: map<str, sf::Sound> - m_fonts: map<str, sf::Font>
+ addTexture(name, path): void + addSound(name, path): void + addFont(name, path): void + addAnim(name, Animation): void + getTexture(name): sf::Texture& + getSound(name): sf::Sound& + getFont(name): sf::Font& + getAnimation(name): Animation&

Textures & Animations



What is a texture?

- A texture is a graphic mapped to a shape
- Can be generated dynamically, or loaded from an existing image file (bitmap)
- A rectangular shape with a texture attached is typically called a ‘Sprite’
- Sprites are very common in games
  - Older computers had sprite hardware

https://www.sfml-dev.org/tutorials/2.5/graphics-sprite.php

SFML Texture Loading

- SFML knows many common image formats, and can load textures from files

```
sf::Texture texture;
if (!texture.loadFromFile("image.png"))
{
    // error...
}
```

Creating a Blank Texture

```
// create an empty 200x200 texture
if (!texture.create(200, 200))
{
    // error...
}
```

Updating Contents of Texture

```
// update a texture from an array of pixels
sf::Uint8* pixels = new sf::Uint8[width * height * 4]; // * 4 because pixels have 4 components (RGBA)
...
texture.update(pixels);

// update a texture from a sf::Image
sf::Image image;
...
texture.update(image);

// update the texture from the current contents of the window
sf::RenderWindow window;
...
texture.update(window);
```

Texture / Shape Size

- Sometimes, the texture will not be the same size as the Entity's shape
- We can specify a sub-rectangle of the texture which will be drawn

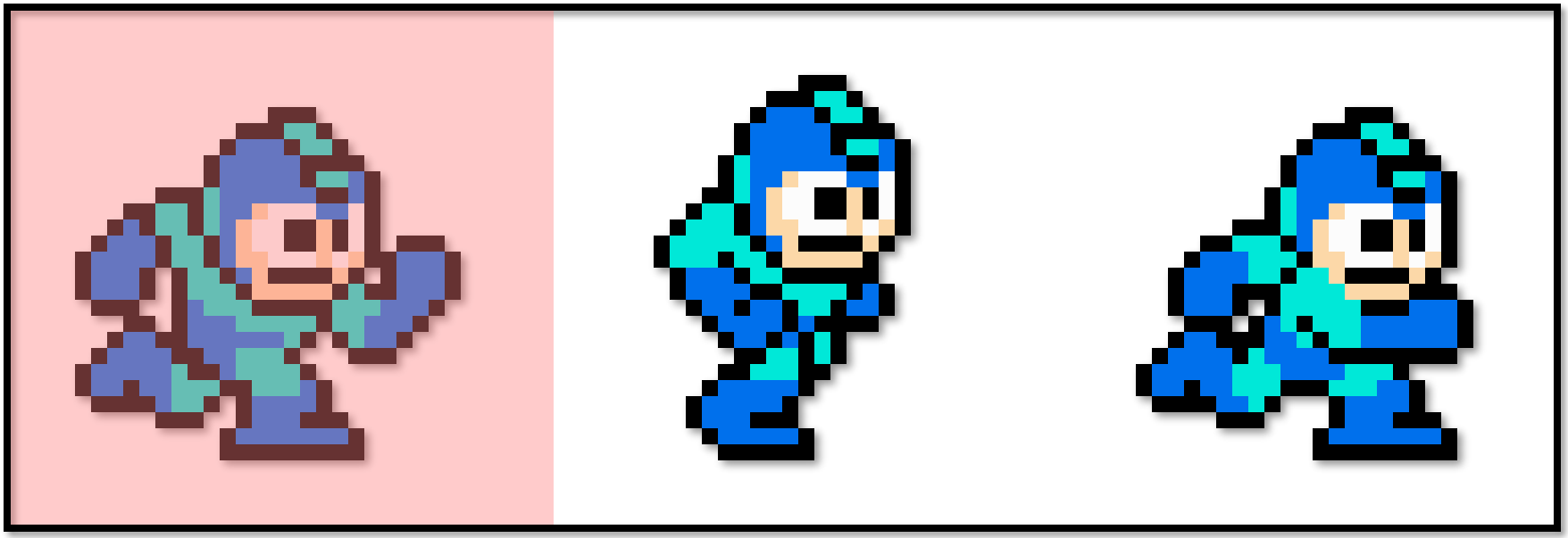
```
// load a 32x32 rectangle that starts at (10, 10)
if (!texture.loadFromFile("image.png", sf::IntRect(10, 10, 32, 32)))
{
    // error...
}
```

Texture / Shape Size

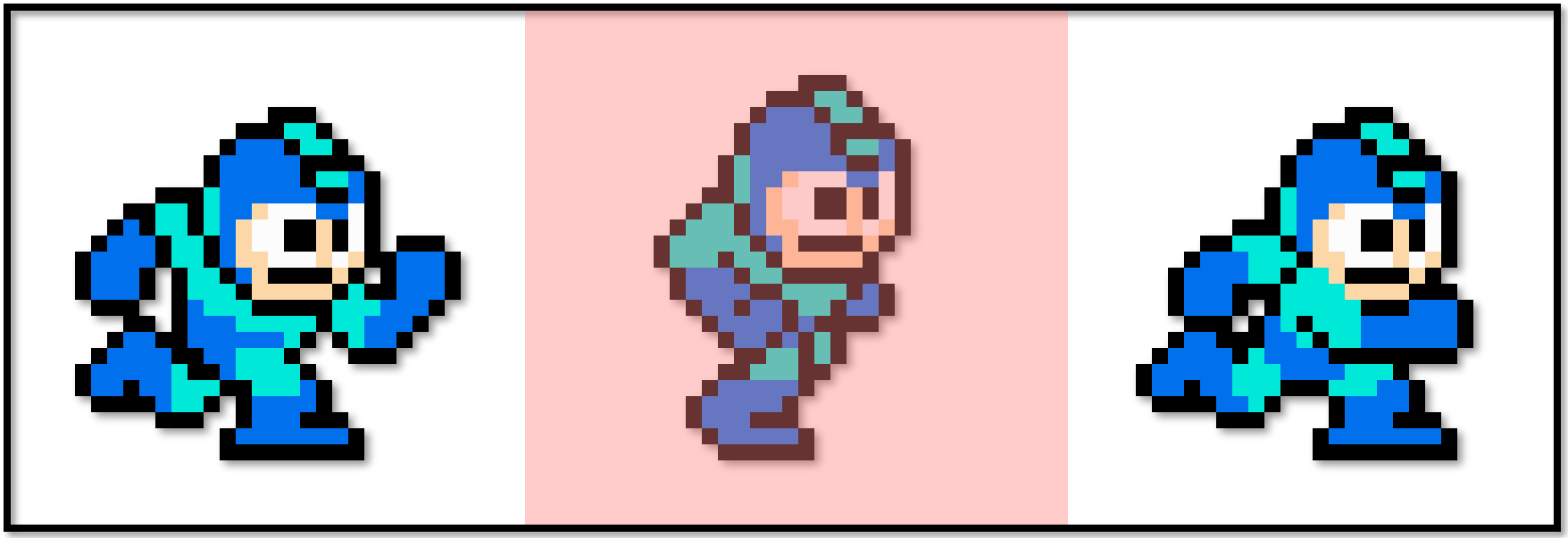
- Sometimes, the texture will not be the same size as the Entity's shape
- Resize the Entity? Affects Gameplay
- Resize the texture? Can be expensive
- We can specify a sub-rectangle of the texture which will be drawn
- Entire texture still loaded, not all drawn

Texture Sub-Rectangle

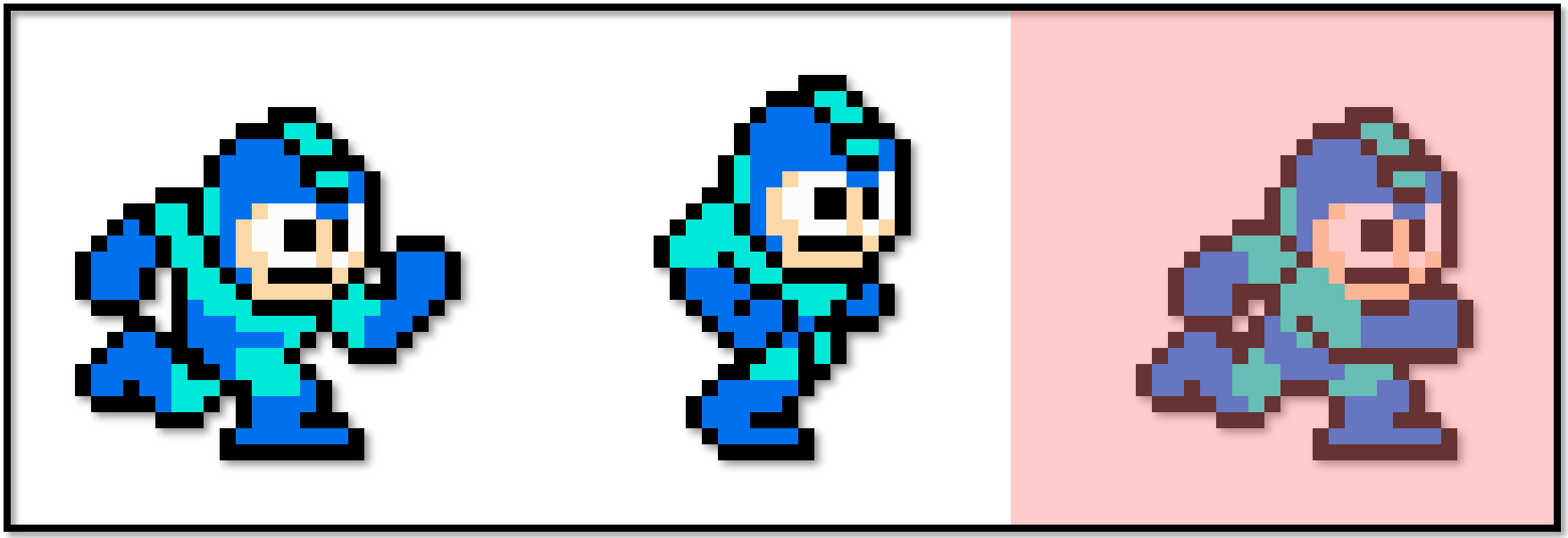
Texture Sub-Rectangle Example



Texture Sub-Rectangle Example



Texture Sub-Rectangle Example



Creating a Sprite

```
sf::Sprite sprite;
sprite.setTexture(texture);
```

```
// inside the main loop, between window.clear() and window.display()
window.draw(sprite);
```

Sprite Sub-Rectangle


```
sprite.setTextureRect(sf::IntRect(10, 10, 32, 32));
```

sf::Sprite Architecture

- Sprite is a very lightweight object
- Consists of vertices, texture pointer, and rectangle of texture to draw

```
////////////////////////////////////////////////////////////
// Member data
////////////////////////////////////////////////////////////
Vertex         m_vertices[4]; ///< Vertices defining the sprite's geometry
const Texture* m_texture;     ///< Texture of the sprite
IntRect        m_textureRect; ///< Rectangle defining the area of the source texture to display
```

Be Careful!

- Sprite store POINTERS to textures

```
sf::Sprite loadSprite(std::string filename)
{
    sf::Texture texture;
    texture.loadFromFile(filename);

    return sf::Sprite(texture);
} // error: the texture is destroyed here
```

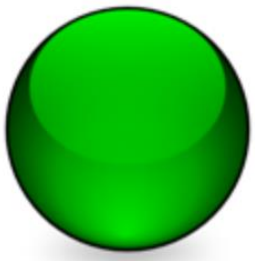
Sprite Colors

```
sprite.setColor(sf::Color(0, 255, 0)); // green
sprite.setColor(sf::Color(255, 255, 255, 128)); // half transparent
```

These sprites all use the same texture, but have a different color:

Original (white)

Grey



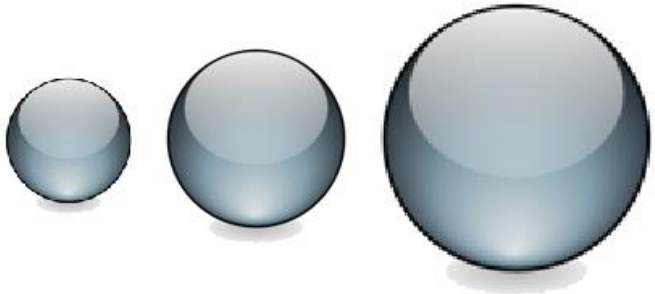
Green

Semi-transparent

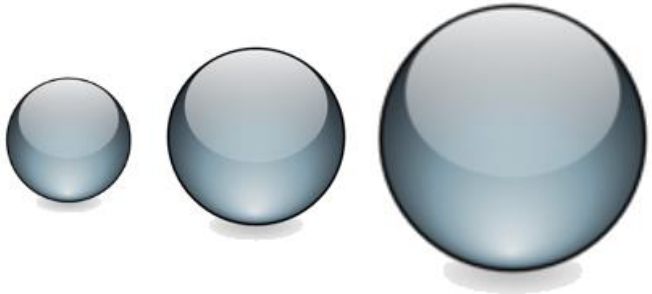
Texture Smoothing

```
texture.setSmooth(true);
```

setSmooth(false)



setSmooth(true)

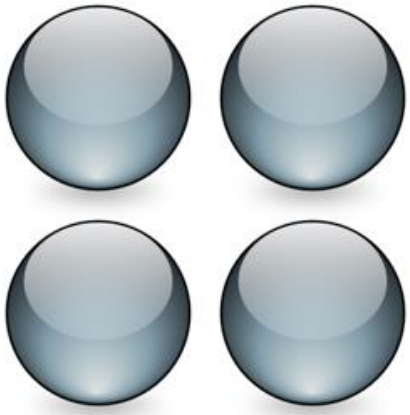


Repeated Textures

```
texture.setRepeated(true);
```

setRepeated(false)

setRepeated(true)



Sprite Transformations

```
// position
sprite.setPosition(sf::Vector2f(10, 50)); // absolute position
sprite.move(sf::Vector2f(5, 10)); // offset relative to the current position

// rotation
sprite.setRotation(90); // absolute angle
sprite.rotate(15); // offset relative to the current angle

// scale
sprite.setScale(sf::Vector2f(0.5f, 2.f)); // absolute scale factor
sprite.scale(sf::Vector2f(1.5f, 3.f)); // factor relative to the current scale
```

Texture Sheets

- Games can have many textures that will be used, can be hard to manage files
- Texture sheets can be used that store many textures within the same image
- Easy way to organize and store textures
- If entire sheet is loaded into graphics memory, less texture swapping occurs

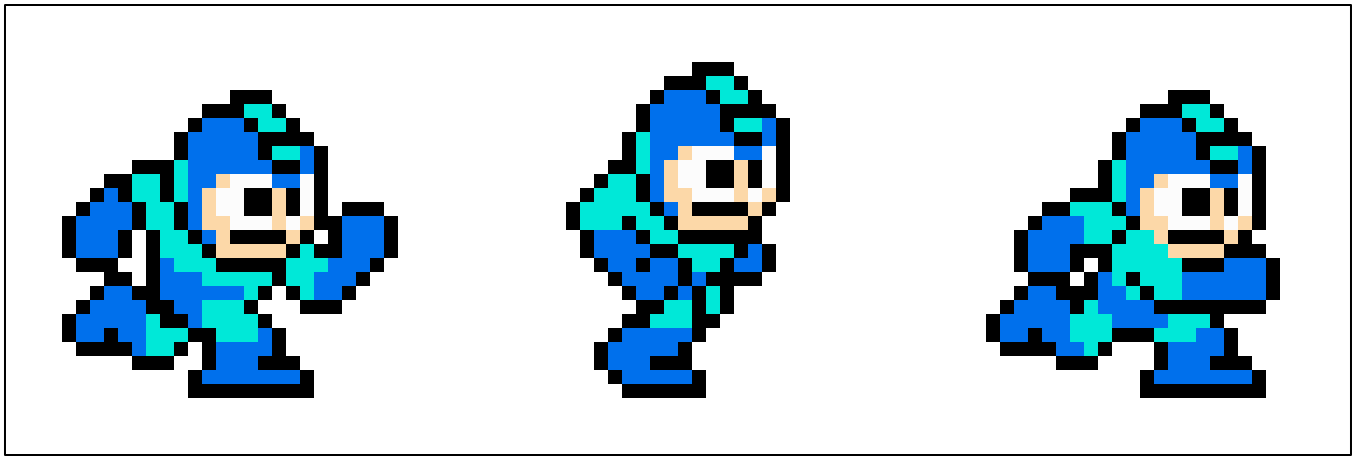
Texture Sheets

- Instead of loading a different texture for every character, pose, tile, you just figure out which sub-rectangle is to be drawn
- Rectangles can be stored as variables, or can be computed dynamically if we construct the sprite sheet cleverly
- Example: `TEX_WALK = IntRect(L,T,W,H)`

Texture-Based Animations

- Animation is a sequence of images that when played quickly appears as motion
- Texture animation can be achieved by quickly displaying different textures
- Bitmap Animation = Raster Animation
  - Made of pixels, rather than vectors
- How to implement this?

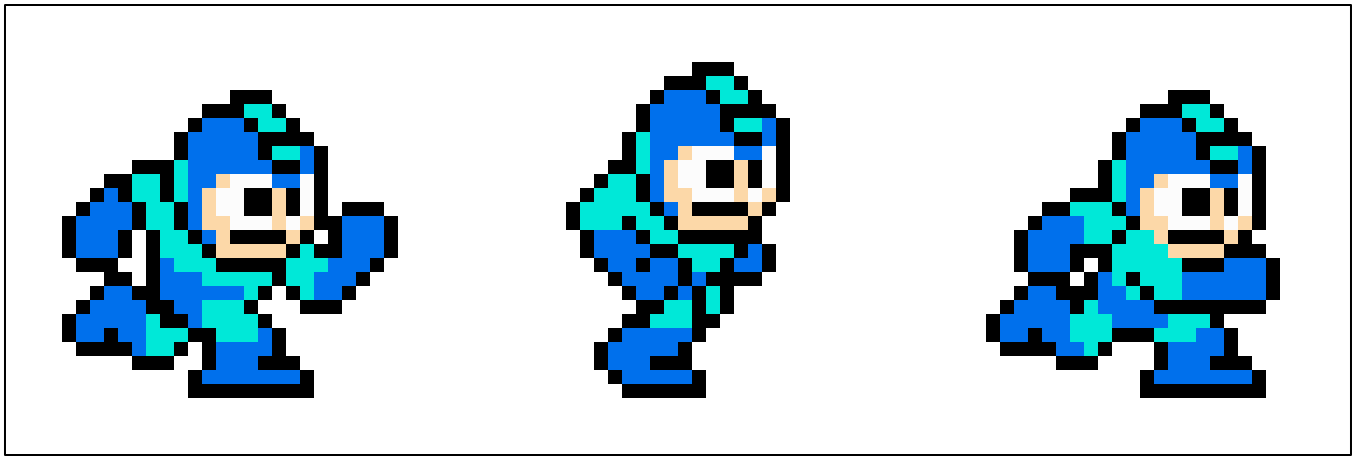
Animations



Animations

Texture Width

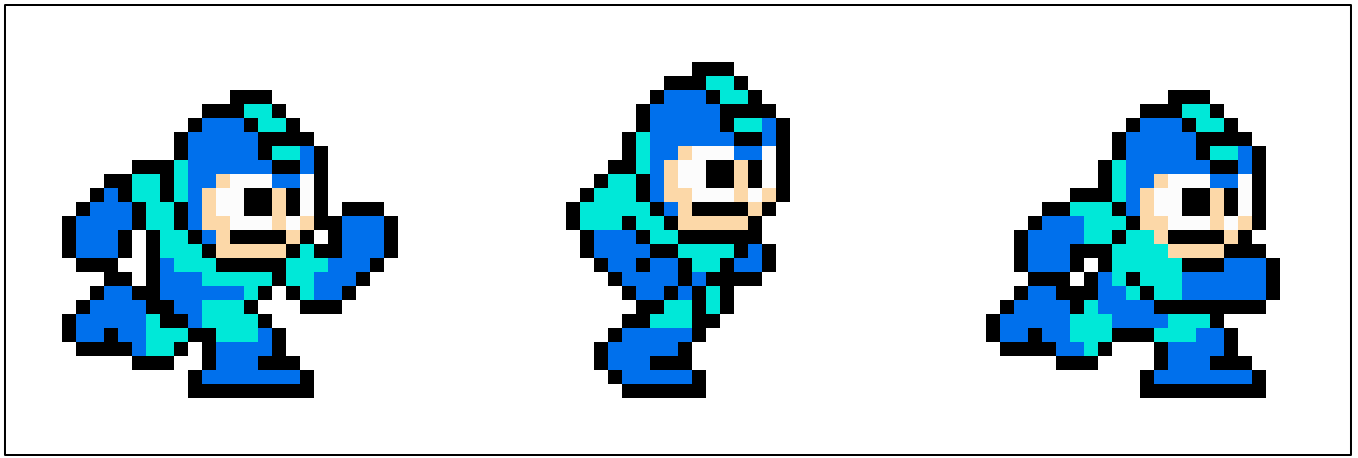
Texture Height



Animations

Texture Width

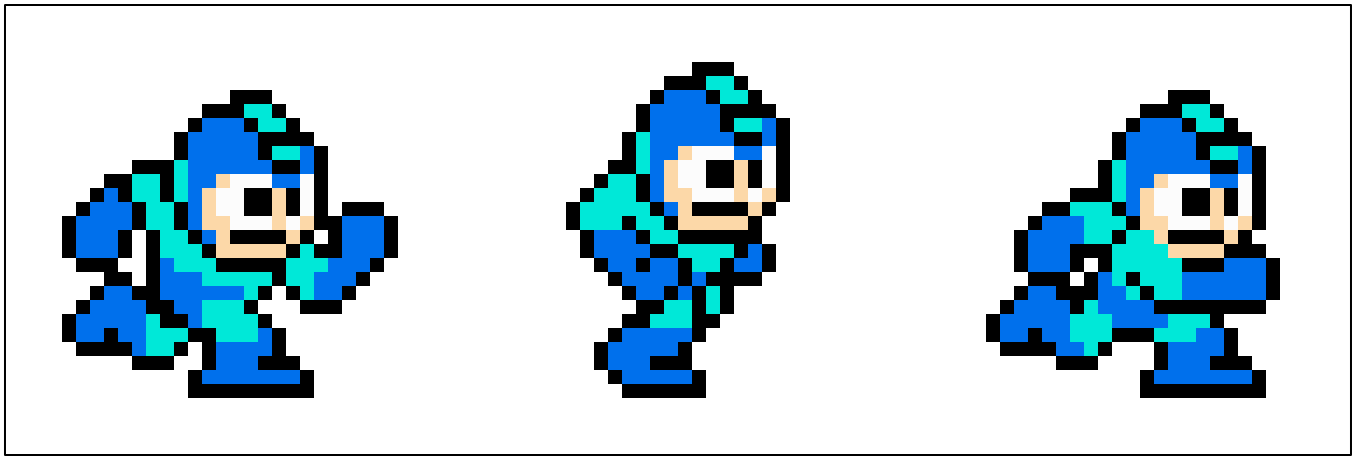
Texture Height



Three Frames of Animation

Animations

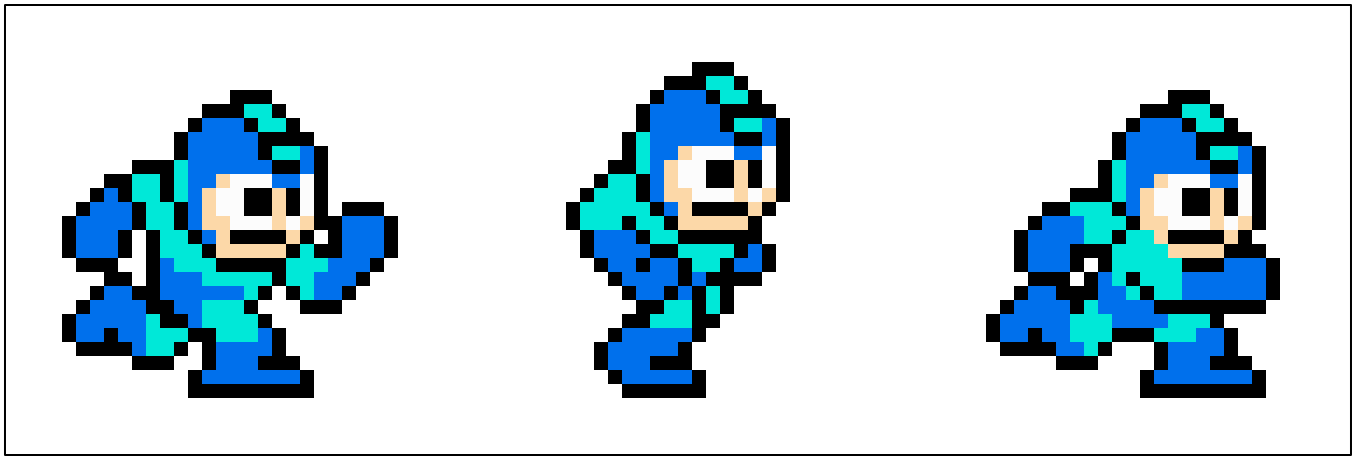
Frame Width (FW) = Texture Width / 3



Three Frames of Animation

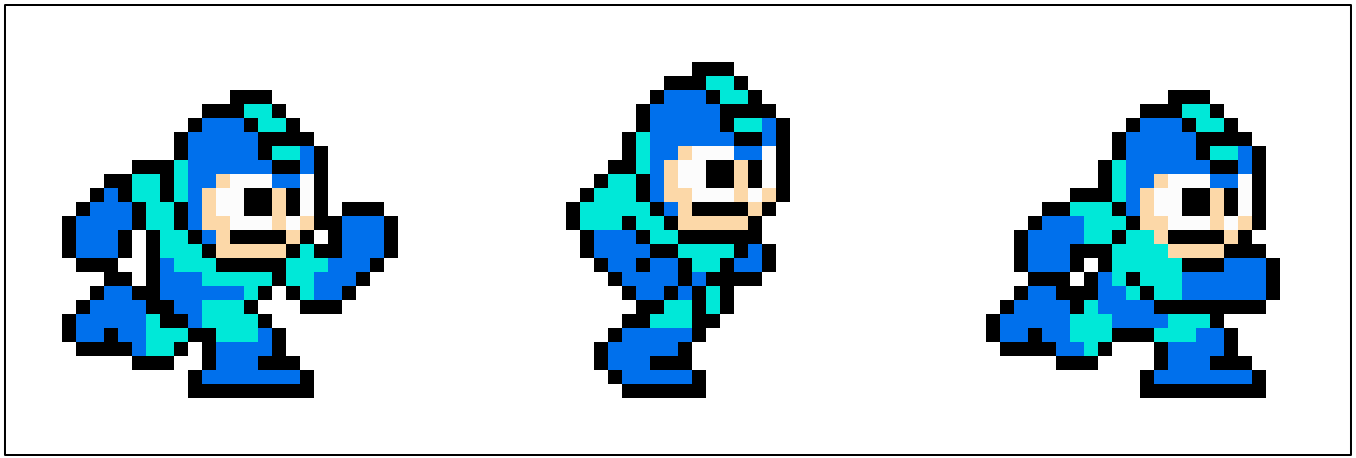
Animations

Rectangle to Draw:
(Left, Top, Width, Height)



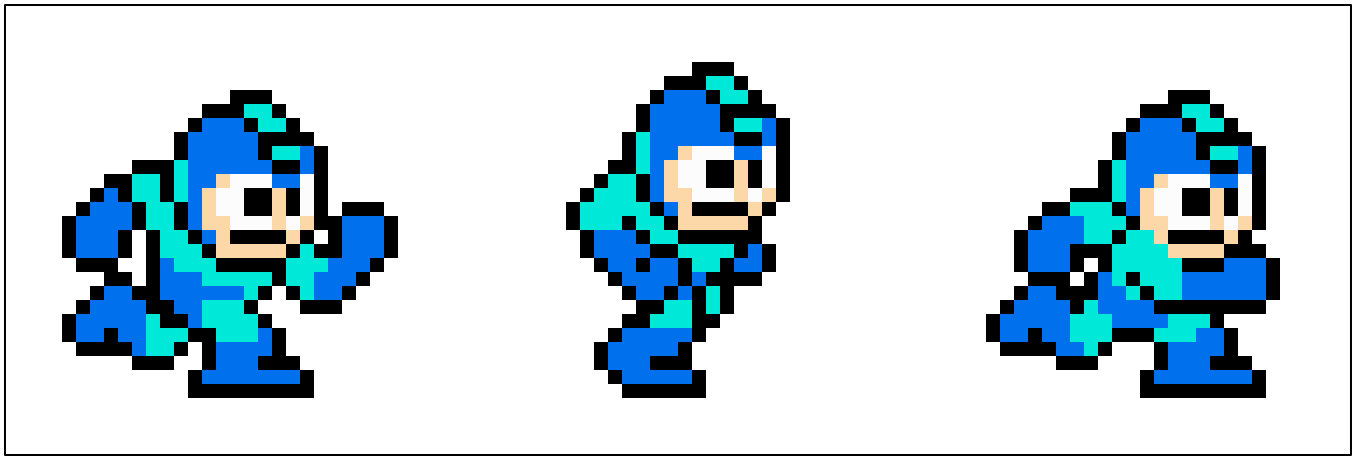
Animations

Rectangle to Draw:
(Frame*FW, 0, FW, FH)



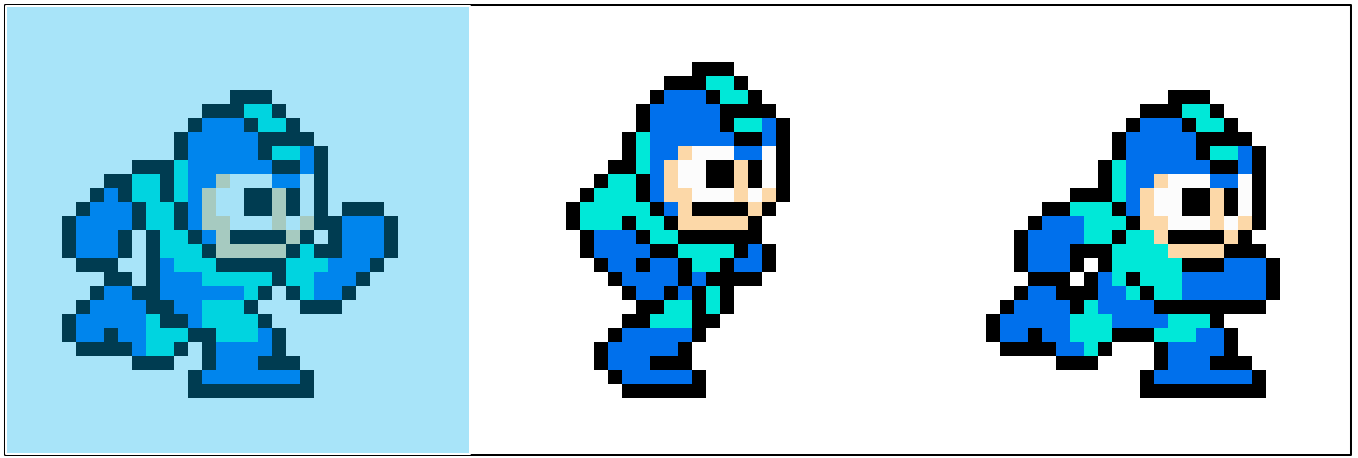
Animations

(Frame*FW, 0, FW, FH)



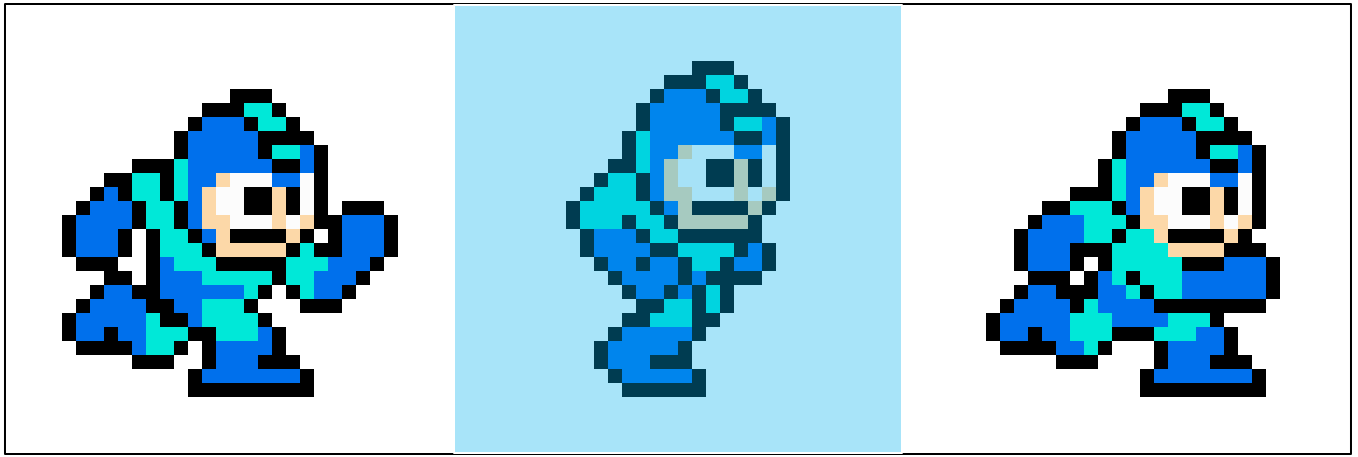
Animations

(Frame*FW, 0, FW, FH)
Frame 0 = (0, 0, FW, FH)



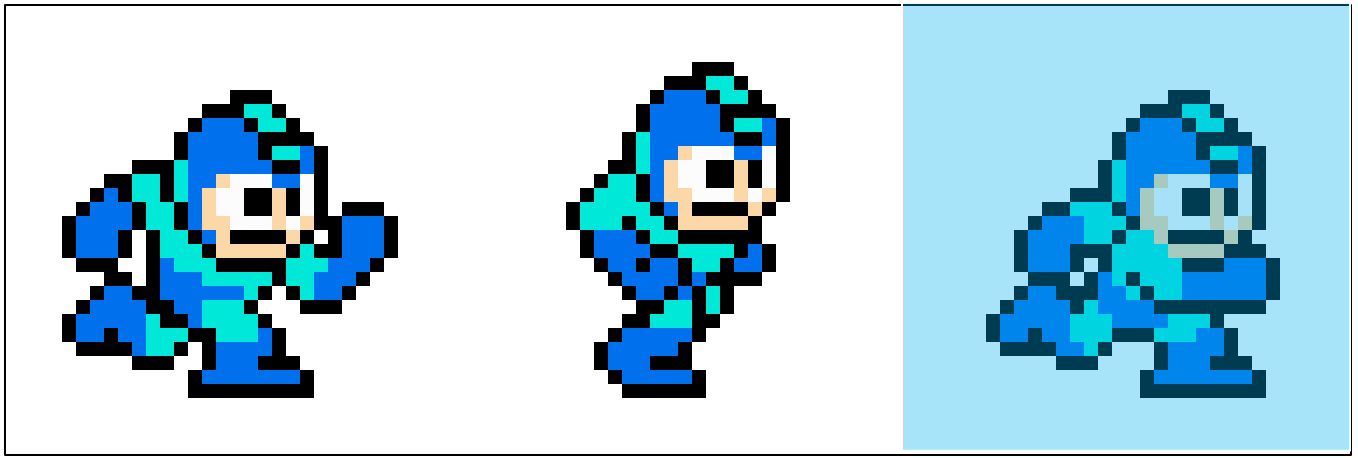
Animations

(Frame*FW, 0, FW, FH)
Frame 1 = (FW, 0, FW, FH)



Animations

(Frame*FW, 0, FW, FH)
Frame 2 = (2*FW, 0, FW, FH)



Animation Implementation

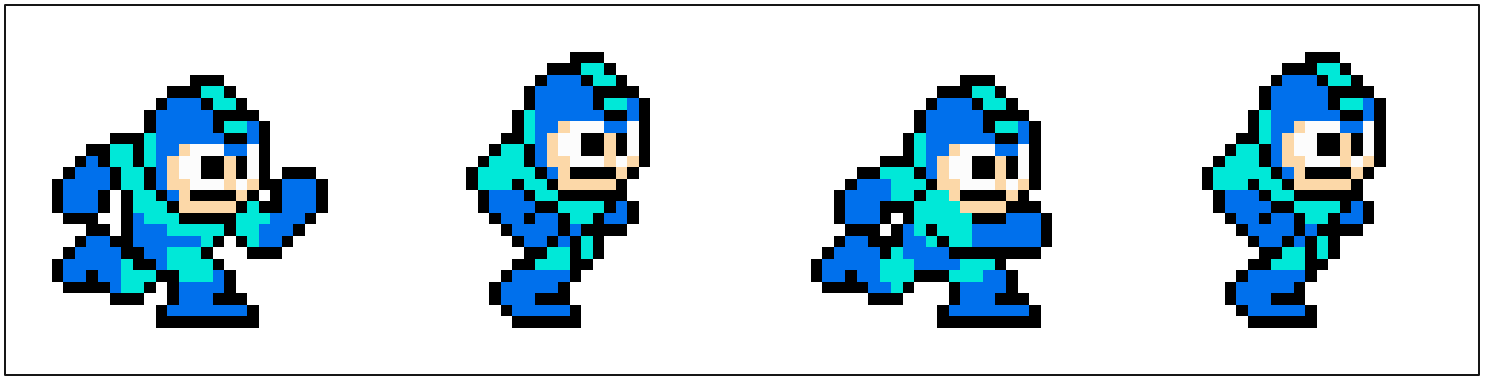
- Animation Class
- Stores all variables related to an animation, with an update() function that provides logic to progress the animation
- CAnimation component stores Animation
- Very slight departure from only ‘pure data’ in component, but very helpful overall

Animation Implementation

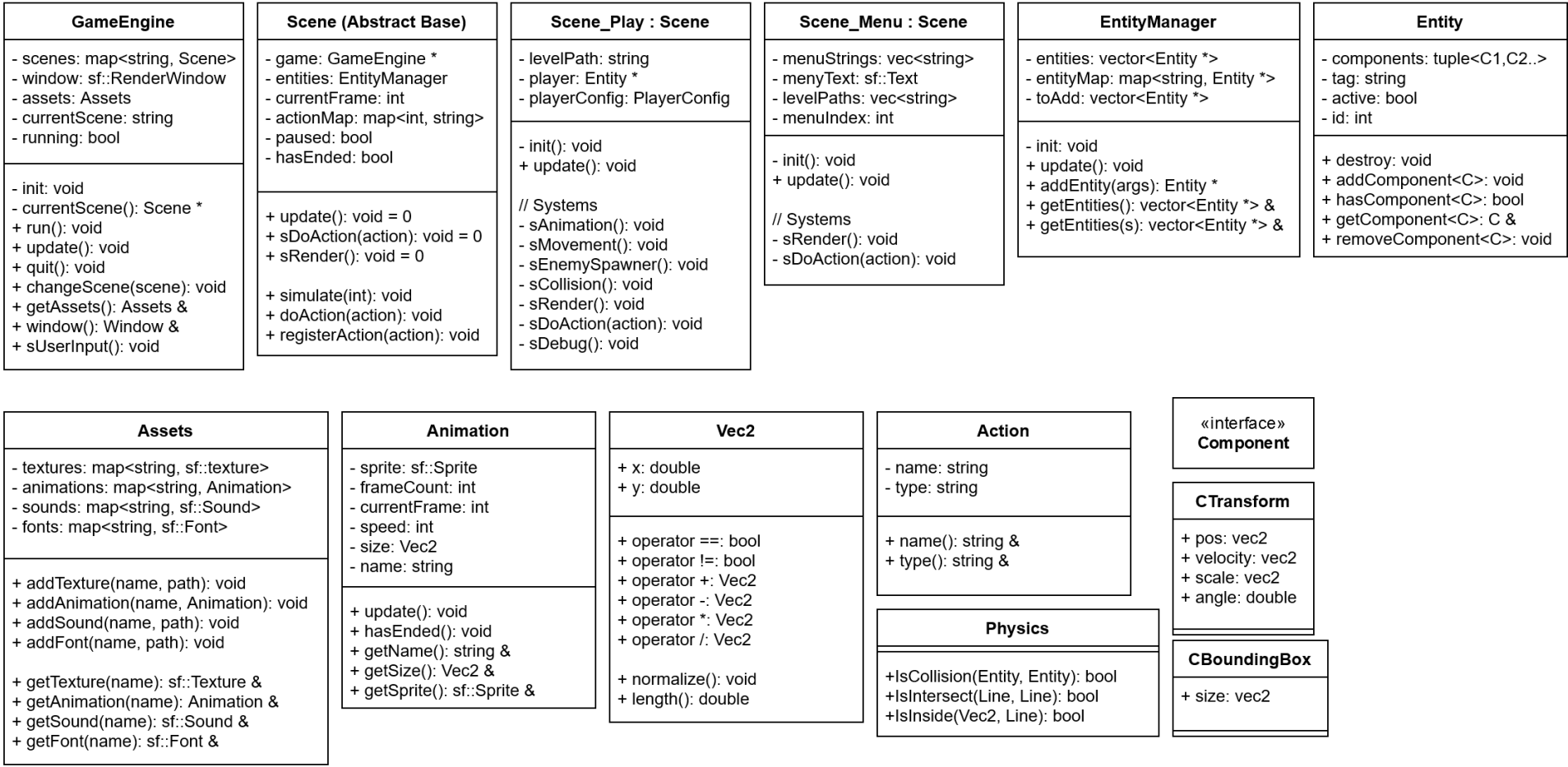
- size_t frameCount, gameFrame
- Vec2 size(texWidth/frameCount, texHeight)
- Variable animationSpeed
- Each update():
  - `gameFrame++`
  - `animFrame = (gameFrame / speed) % frameCount`
  - `rectangle = (animFrame*FW, 0, FW, FH)`
  - `sprite.setTextureRect(rectangle)`
- We will make Animation class for this

Animation Specification

- Animation details specified in assets file
- Example Assignment 3 ‘Run’ Animation
  - `Texture TexRun images/megaman/run64.png`
  - `Animation Run TexRun 4 10`



COMP 4300 - Game Engine Architecture



Animation Video(s)

The Importance of Keyframes / Run Cycles

https://www.youtube.com/watch?v=fJosaT1sCfs

How Old School Graphics Worked

https://youtu.be/Tfh0ytz8S0k